CAMPUSHARE

Final Pre-Build Audit & Implementation Plan

Version 1.2 | 28 October 2026 | Solo developer | Code Conqueror India 2026

Master source: the official “Unlock Underused Equipment and Spaces” problem statement supplied for the hackathon. This document supersedes earlier plan details where it explicitly simplifies them; it does not override the organiser’s requirements.

Executive verdict

APPROVED TO BUILD, WITH FINAL CLARIFICATIONS. CampusShare is the right concept and maps to the official brief. This final compliance pass locks five details before coding: supervision must be an approval gate rather than metadata; conflict handling must offer a clear next action; overdue physical custody must block later approvals; reminder timing must fit short bookings; and fairness limits must use one consistent count-based policy. These are clarifications, not scope expansion.

Final changes in v1.2: supervision gate enforced; conflict recovery made explicit; overdue physical custody blocks re-booking; due/overdue notices use one verified 15-minute scheduled job; fairness limits made consistent and count-based.

The product in one paragraph

CampusShare is a single-campus resource-sharing desk for students and project groups to discover, request and responsibly use portable, non-hazardous equipment and a small number of reservable project spaces. Owner-controlled approval, explicit eligibility, simple fair-use caps, recorded handoffs/returns, incident review, in-app reminders and an append-only Trust Record make the full journey understandable and accountable.

One non-negotiable release condition

A judge must be able to use the live site without the developer, complete one persisted listing-to-return journey for equipment and see the space-use path, inspect both start/end condition records and the Trust Record, and test a blocked eligibility/supervision case, an overlap conflict and an overdue-custody block. A landing page or disconnected prototype screens do not pass.

1. Final review of the previous plan

Decision	Keep / adjust	Reason
Product concept	KEEP	It addresses the brief’s trust, fairness, custody and access-control requirements rather than only displaying bookings.
One-campus scope with equipment + non-hazardous spaces	KEEP	Specific enough for a solo build while visibly covering both “equipment” and “spaces”.
Four role types: borrower, owner, manager, admin	KEEP BUT SIMPLIFY UI	The brief asks for role-based access for all four. Use one shared app shell with role-aware tabs/actions, not four independent applications.
Extensive fairness controls	SIMPLIFY	Use a small, visible set of request/use caps plus FIFO waitlisting. Drop rolling-hour calculations and multiple overlapping quota formulas.

Decision	Keep / adjust	Reason
Separate extension_requests database table	REMOVE	Keep the latest extension request fields on the booking and preserve every extension action in the activity log. This avoids an extra table without losing history.
Many individually written audit/notification operations	SIMPLIFY	Use a small set of trusted action functions plus shared database helpers/triggers so state changes, activity events and notices commit together.
Cron + fallback reminder system	KEEP, ONE JOB	Use one database job for due-soon/overdue notices; also derive a visible due/overdue state in the app. Do not add email/SMS/push.
AI, QR, photos, external calendar, payments	OUT OF MVP	These do not strengthen the main workflow enough to justify setup and failure risk in 48 hours.
20 independent acceptance tests, all treated as separate P0 tasks	REPACKAGE	Keep every requirement represented, but test them through about 12 grouped scenarios so the checklist is manageable.
Final compatibility plan v1.2	USE AS LOCKED BUILD AUTHORITY	This version includes the final requirement-by-requirement compatibility fixes and acceptance evidence; use it for implementation.

Research basis: I-STEM already supports equipment discovery and requests while host institutions retain approval authority; resource booking and asset-management systems already cover many individual functions. Do not claim those functions are unique. Our product bet is an easier-to-follow, fully joined-up workflow.

2. Official problem statement coverage matrix

Each requirement below has a concrete, testable MVP interpretation. The choices below are designed to satisfy the submitted problem statement while keeping implementation suitable for one developer.

Official requirement	MVP implementation	Acceptance evidence
Resource listing: location, availability, access conditions, rules	Owner creates a unique equipment item or reservable non-hazardous space with location, availability, duration limit, usage/access rules, eligibility/training requirements and a supervision-required flag. A resource requiring supervision must have an authorised supervisor confirmed before approval.	Owner creates and publishes a camera kit and project room; details are visible. A supervised seed resource cannot be approved without a named authorised supervisor.
Search/filter/intelligent matching	Case-insensitive text search plus type, category, location and available-slot filters; an eligible-only filter is optional.	Search terms and filters change results correctly; no AI matching required.
User and owner profiles / verification	Seeded fictional accounts; manager verifies campus affiliation and owner authority.	Unverified owner cannot publish; unverified borrower sees a clear blocker.
Eligibility for training/qualifications/membership	Resource defines a required training code or allowed audience; manager-maintained status and expiry are checked on request and rechecked on approval.	Missing or expired training prevents owner approval and explains what is missing.
Booking, approval/rejection/waitlist	Persist each request. Owner decides approve/reject/waitlist with an explicit decision explanation; rejection and waitlisting require a meaningful human reason. Waitlist order is FIFO by default, with any later selection reason recorded.	Both parties can see status and reason; waitlist position is visible; approval records why the request met requirements.

Official requirement	MVP implementation	Acceptance evidence
Scheduling conflicts	Validate date order, operating window, maximum duration, scheduled overlap and active custody. On conflict, explain it and let the user choose another time or join the FIFO waitlist. While a unique item is Checked Out/In Use or Return Pending with no actual hand-back, block all other approvals even if planned end time passed.	Overlapping reservations cannot both be approved; adjacent intervals are allowed; an overdue item cannot be re-booked until hand-back is confirmed; user gets a clear alternative action.
Check-in/check-out and return confirmation	Same lifecycle for both types. Equipment has condition/accessory checklist; spaces have a brief room/workspace condition checklist. Both borrower and owner/custodian acknowledge start and end handoff.	A reservation remains occupied until end condition is submitted and owner confirms return/hand-back.
Before/after condition documentation	Required structured condition code and short checklist/note at start and end. Photos are not required.	Both condition records persist; a worsened or damaged result opens an incident.
Damage, delay and conflict handling	Borrower or owner can report damage, delay or a suspected scheduling conflict; incidents are linked to the resource/booking, reviewed by an authorised human and closed with a resolution note. Overdue/no-show needs human confirmation.	A reported conflict/damage/delay appears in the authorised review queue; involved users see status and resolution while unrelated borrowers cannot view private incident details.
Notifications for approval, reminders, cancellation, overdue	Persistent in-app inbox on decisions, cancellations, extensions and incidents; one scheduled job running every 15 minutes creates due-soon notices within 60 minutes of planned end and overdue notices if hand-back is unconfirmed.	Borrower and owner/custodian receive due-soon/overdue notices, deduplicated by booking + notice type + planned end timestamp; extensions create a fresh reminder cycle. Decisions and operational events create persistent notices; scheduled execution is verified.
Fair access / avoid monopolisation	Individual: maximum 3 approved uses starting in a rolling 7-day window and 2 open booking items at once. Pre-approved group: maximum 5 uses in a rolling 7-day window and 3 open items. Open items include pending, waitlisted and approved-but-not-started bookings; group use also counts against the submitting member. FIFO waitlist; exceptions require role + reason.	Remaining limits are visible; cancelled/rejected requests do not consume use quota; the cap blocks with an explanation; group use counts against both group and individual quota; any override is logged.
Role-based access: user, owner, manager, admin	Distinct authenticated demo roles with one shared interface. Owner is resource-scoped; manager is campus-scoped; admin manages roles/policy.	Direct table/RPC attempts that exceed a role fail, not merely hide buttons.
Auditable decisions/actions	Trust Record for resource and booking events: actor, timestamp, action, old/new status and reason. No normal client update/delete for old events.	Approve/reject/cancel/extend/check-out/return/incident events remain after refresh.
Privacy/security	Synthetic data only; minimum profile fields; RLS and least-privilege grants; no public exposure of private purposes or incidents; no secrets in client.	Negative tests prove a borrower cannot read unrelated private records or promote their role.
Human oversight	Owner remains decision-maker. Managers review eligibility/exceptions/incidents. No automated penalties or unexplained AI decisions.	All final access/incident outcomes are attributable to an authorised human.
Meaningful AI use	No runtime AI in MVP. Deterministic eligibility, conflict and fairness rules are explainable. AI development assistance is disclosed.	Product works without an external model/API or model quota.
End-to-end usable outcome	One complete persisted journey with a denied/conflicting case; live URL plus source, instructions, report and disclosure.	Judge can follow README/test instructions without developer assistance.

3. Exact product scope: include / exclude

Included in the MVP

- One fictional campus and one institution; no multi-tenant provisioning.
- Two resource types: unique portable equipment units and a small number of non-hazardous project/meeting spaces. Seed 6–10 credible fictional resources across both types.
- Owner listing/editing, publish/pause/archive, availability windows, duration limit, location, rules and eligibility flags.
- Search, filters, resource detail and a clear eligible/not-eligible explanation.
- Authenticated demo accounts and four distinct roles. No public self-registration required for judges; synthetic users are prepared in advance.
- Booking request, eligibility check, owner decision with reason, FIFO waitlist, cancellation and extension request/decision.
- Database-level overlap prevention, usage caps and a small registered demo project group.
- Start/handoff condition, end/return condition, owner acknowledgement and incident review.
- Persistent in-app notification inbox, due-soon alerts when a booking is within 60 minutes of its planned end, overdue notices, and one scheduled job running every 15 minutes with deduplication and a repeatable test path.
- Trust Record / activity timeline, readable mobile/laptop UI, README, test instructions, project PDF and reuse/tool disclosure.

Explicitly excluded

- Payments, security deposits, credits, refunds, auctions or paid priority. Demo resources are explicitly free/institution-sponsored; the app must not imply that deposits or payments are supported.
- Government-ID checking, college SSO/SIS, real training document upload, biometric checks or any claim of production institutional verification.
- Hazardous machinery, chemicals, clinical/regulated equipment, clean-room and high-risk fabrication workflows.
- Photos/video/OCR/computer vision for damage; structured condition labels and notes are sufficient for the demo.
- Email, SMS, WhatsApp, push notifications and calendar integrations; use in-app notifications.
- AI matching/agents, automated penalties, automatic blame assignment, predictive ranking or opaque priority scores.
- Native mobile app, maps, recurring reservations, multi-resource kits/quantities, analytics suite, public user-generated groups, advanced admin portal or production-grade dispute/legal workflow.

If the core workflow is behind schedule, cut visual polish and optional reporting, not any of the explicit problem-statement outcomes. For example, no email/SMS is acceptable only because persistent in-app notices and due/overdue reminders remain implemented.

4. Users and permissions

Role	Can do	Cannot do
Borrower	Browse published resources; see rules and eligibility; request/cancel; request extension; check out/submit end condition when approved; report incident; read own booking, notifications and Trust Record.	Cannot publish, approve own booking, edit verification/role fields or read other users' private booking/incident details.
Owner / custodian	Create/edit own resource; publish after owner verification; review bookings for own resource; approve/reject/waitlist with explanation; assign/confirm an authorised supervisor when needed; acknowledge handoff/return; review related incidents; cancel a resource booking with reason.	Cannot access another owner's queue, silently bypass fairness controls or edit/delete audit history.

Role	Can do	Cannot do
Manager / coordinator	Verify affiliation/training/owner status; handle pending eligibility; review incident; manage campus-level exception/queue override with reason; inspect campus audit records.	Cannot assign global admin role or erase history; sees only information needed for operational decisions.
Admin	Assign/revoke demo roles; set visible default policy; pause/archive unsafe or unavailable resources; manage seed/demo records; inspect system-level history.	Cannot rewrite/delete audit history or use confidential real records.

Keep all roles in one application shell with conditional navigation/actions. Role names and underlying authorization remain distinct even if the visual layout is shared. For the demo, use separate seed accounts rather than an insecure client-side role switcher.

5. The final user journeys

A. Main success journey

1. Owner signs in and creates a resource with location, availability, duration, rules, eligibility/training requirement and a supervision flag. Manager verifies owner authority; owner publishes. If supervision is required, an authorised owner/manager must assign or confirm a named eligible supervisor before approval; use the owner/custodian as the supervisor for the simple demo case.
2. Borrower searches, opens details and sees availability/rules/eligibility before submitting a time range, short purpose and optional approved group.
3. System checks affiliation, training, resource status, hours/duration, quotas, duplicates and conflicts. If verification is incomplete, request is saved as Pending Verification without reserving a slot; otherwise it becomes Pending Owner Review. A supervised resource cannot be approved until the required supervisor is confirmed.
4. Owner reviews the request. Approval rechecks all rules at that moment and records an explicit explanation (default: eligibility passed, slot available and owner approved); rejection and waitlisting require a human-readable reason. Waitlisting is allowed only for otherwise-eligible requests. FIFO is the default review order; a later request may be chosen only with a recorded reason. No waitlisted request is auto-approved.
5. At start, borrower and owner/custodian record and acknowledge the initial condition together. For equipment, record condition and included accessories; for a space, record room/workspace condition and relevant fixtures. Booking becomes Checked Out/In Use.
6. Borrower submits the end condition; owner/custodian confirms return/hand-back. For a space, use end-of-use and room-condition confirmation. If condition worsened, the system creates an incident instead of silently closing it.
7. Authorised manager/owner reviews the incident, records a resolution note and closes it. Both parties can read the Trust Record showing the full sequence.
8. Judge refreshes or opens a second session; records and statuses remain persisted.

B. Negative / exception journeys

- Missing/expired training: clear eligibility blocker; owner cannot approve; manager updates the synthetic qualification; user rechecks eligibility; transition is audited.
- Conflict: a second overlapping approval fails with a clear explanation. The borrower can choose a different time or join the FIFO waitlist; the system never silently moves another booking. If any resource remains Checked Out/In Use or Return Pending after its scheduled end, all new approvals for that resource stay blocked until actual release/hand-back is confirmed.
- Fairness cap: request is blocked with used/allowed count. A documented override requires an authorised role and reason, and appears in history.
- Owner rejects: reason required; borrower gets a notice and sees the reason in the booking record.
- Condition mismatch/overdue: create incident or overdue notice; no automatic fine/suspension; authorised human reviews the matter.

6. Business rules and state model

Resource state

State	Meaning	Rule
Draft	Only owner/authorised manager sees it	Not discoverable/bookable.
Published	Eligible users can discover it	Owner must be verified; required fields complete.
Paused	Temporarily unavailable	No new requests; existing bookings are reviewed, cancelled with reason and notified if necessary.
Archived	No longer offered	No new requests; historical bookings remain readable to authorised parties.

Booking status

Status	Meaning	Important rule
Pending Verification	Affiliation/training/authorisation incomplete	Does not reserve a time slot; show exact missing requirement.
Pending Owner Review	Eligibility passed; waiting on owner	Does not reserve slot; approvals must recheck all rules.
Waitlisted	Eligible, but owner/slot unavailable	No reservation promise; queue order visible; no auto-approval.
Approved	Owner approved; slot is reserved	Must remain conflict-free.
Checked Out / In Use	Equipment handed off / space use started	Resource remains occupied.
Return Pending	End condition entered; custodian not yet confirmed	End condition entered; custodian has not confirmed hand-back. Resource stays blocked from all new approvals until actual hand-back, even if the planned end time has passed.
Returned	Custodian confirmed hand-back	Terminal booking status; linked incident may remain open.
Rejected	Owner denied request with reason	Terminal; new request may be submitted if eligible.
Cancelled	Request/booking cancelled with actor and reason	Terminal; slot is released; waitlist is re-evaluated.
Overdue flag	End time has passed while in use	Derived indicator/notification, not an automatic accusation or penalty.

Locked policy defaults (visible as demo policy)

Rule	MVP default	Why / behaviour
Individual use cap	Maximum 3 approved uses whose start time falls in a rolling 7-day window; maximum 2 open booking items at once (pending verification, pending owner review, waitlisted, or approved but not started).	Count approved, in-use, return-pending and returned uses with start time in the window; exclude cancelled/rejected/waitlisted/pending items from use count. Open items are counted separately against the 2-item limit.
Project group cap	Maximum 5 group uses in a rolling 7-day window; maximum 3 open group booking items at once.	Only pre-seeded/manager-approved groups. A group booking counts against both the group use/open-item limit and the submitting member's individual limits.
Open-item definition	Waitlisted entries are included in the open-item limit; no separate waitlist quota is added.	Prevents queue hoarding without introducing another overlapping limit.

Rule	MVP default	Why / behaviour
Duration	Default max 4 hours for equipment and 2 hours for spaces; owner may set a lower cap.	Avoids long monopolising reservations and is easy to explain.
Queue order	FIFO by created timestamp; request ID breaks ties.	Owner normally reviews the first eligible waiter when a slot opens; selecting a later request requires a recorded reason and can be reviewed by a manager. Never auto-approve a waitlisted request.
Eligibility	Affiliation must be verified; required training must be verified and unexpired; any required supervisor must be assigned/confirmed before approval.	Eligibility and supervision are checked at request and again before approval.
Interval math	Half-open [start, end): adjacent bookings may meet at the boundary.	No accidental conflict when one booking ends exactly as another begins.
Time zone	Store timezone-aware UTC instants; display Asia/Kolkata.	Consistency in conflict and deadline checks.

These numbers are prototype policy, not an assertion about any real college. Display them as “Demo Fair-Use Rules.” Use booking counts, not hour-based caps. Open-item limits include pending verification, pending owner review, waitlisted, and approved-but-not-started bookings; no separate waitlist quota is needed. Do not add rolling-hour quotas or overlapping quota formulas.

Extension and cancellation

- Borrower can cancel pending requests; approved bookings can be cancelled before check-out. Cancellation requires actor and reason; release the slot and notify affected parties.
- After check-out, do not silently cancel. Complete return/hand-back or create an incident.
- An extension is a request for a new end time plus reason. The owner decides; approval rechecks duration and scheduled conflicts. If the item is physically still checked out/return-pending, it remains unavailable to every other request until actual hand-back, even after the old planned end time. Latest extension fields live on the booking; every request/decision is retained in the audit log.
- No-show/overdue is a signal, not a penalty. Authorised owner/manager confirms the incident with a reason.

7. Minimal architecture

Layer	Choice	Implementation rule
Frontend	Vite + React + TypeScript	One SPA, shared shell, role-aware navigation and typed domain/status values. No Next.js or custom server unless unavoidable.
UI	Tailwind CSS + selected shadcn/ui + Lucide	Add only cards, forms, dialogs, status badges, table/list, tabs and toasts needed for the journey.
Backend/data	Supabase Auth + PostgreSQL	Use one hosted data store; no separate FastAPI service in MVP.
Authorization	RLS + least-privilege table grants	All exposed tables have RLS and explicit grants; sensitive roles/statuses cannot be client-edited.
Critical actions	Small set of database RPC functions	Use trusted functions for booking request/review, lifecycle action, verification and incident resolution; validate `auth.uid()`, role and state. Set secure search_path and restrict execution grants.

Layer	Choice	Implementation rule
Audit/notifications	Shared helper/triggers within the transaction	Every successful transition adds one event and relevant in-app notice in the same transaction. Failed transitions add no success record.
Conflict safety	Postgres exclusion constraint preferred	Prevent overlapping scheduled reservations with a PostgreSQL exclusion constraint where available. Approval RPC also blocks any resource with an active physical checkout/space use or return-pending booking lacking actual release confirmation, even when the planned end time is past. Test extension/availability early; keep a tested transaction/lock fallback.
Deployment	Static host (Vercel or Render) + Supabase	Deploy a blank shell early, then redeploy from the same repo. Never change platform late without a blocking reason.

Minimal data model

Table	Key purpose / fields
profiles	auth user ID, display name, primary demo role, campus affiliation status, owner verification status, verifier and timestamps.
qualifications	user ID, training/credential code, status, expiry, reviewer and verification timestamp.
project_groups	ID, name, verified status, reviewer; seeded groups only.
group_members	group ID, user ID, status, manager-added/approved by.
resources	type, category, title, summary, safe location label, owner, state, availability window, max duration, rules, training requirement, supervision flag, cost policy.
bookings	resource/requester/optional group, interval, purpose, status, supervisor_user_id when required, actual_start_at and actual_release_confirmed_at, eligibility snapshot, decision reason, timestamps, latest extension request/decision fields.
condition_reports	booking, phase (start/end), condition code, accessory checklist, note, reporter, acknowledgement and timestamp.
incidents	booking/resource, type, description, status, reporter, reviewer, resolution note and timestamps.
notifications	recipient, type, related entity, message, created/read timestamps, unique dedupe key.
activity_events	actor and role snapshot, entity/action, previous/new state, reason, safe metadata, timestamp; normal client cannot update/delete.

Simplifications from the earlier schema: remove the separate `user_roles` table for the one-role-per-demo-account prototype and remove the separate `extension_requests` table. Extension history stays in activity events. Do not store documents/photos or sensitive IDs.

Critical database behaviour

- All critical state changes are transactions: validate current role/owner/scope/status and business constraints, update the record, add the audit event and relevant notification, then commit. Resource publish/pause/archive, affiliation and training verification, supervisor assignment, decisions, cancellations, extensions, handoffs, returns and incidents are included in the activity history.
- Use an exclusion constraint on resource ID plus the [start,end) range for overlapping scheduled intervals that reserve capacity (Approved, Checked Out/In Use and Return Pending); pending requests/waitlist do not reserve a

slot. Because a range constraint alone cannot represent equipment not returned or a space not released after its planned end, the approval RPC must also reject approval whenever a resource has a Checked Out/In Use or Return Pending booking without actual release/hand-back confirmation, regardless of the requested slot. Release that block only after authorised confirmation.

- RLS is not replaced by frontend filtering. Test table grants as well as policies; use only a publishable key in frontend. Never expose Supabase service-role/secret keys.
- Keep incident/private purpose/training details visible only to involved parties and authorised manager/admin. Public catalogue exposes no private contact/security details.
- One Supabase Cron job runs every 15 minutes and creates a due-soon notice when the planned end is within the next 60 minutes and an overdue notice after the end time if hand-back is unconfirmed. Notify both borrower and owner/custodian. Deduplicate by booking + notice type + planned end timestamp, so the same event is not sent repeatedly and an approved extension can create a new reminder cycle. Display due/overdue status in-app. Test with seeded dates and inspect Cron run history. A UI badge alone is not a working reminder.

8. Screens: one shell, no overdesign

Screen / route	What it must show	Role visibility
Sign in / demo instructions	Simple login; clear note that accounts/data are fictional. Demo credentials supplied in judge instructions.	All, signed out.
Resource catalogue	Search/filter cards, availability, location, owner verification, conditions and eligibility badges.	Borrower/owner/manager/admin; only published resources public to signed-in app users.
Resource detail + request form	Full rules; availability; eligibility/supervision explanation; date/time/purpose; group if applicable; submit or clear blocker.	Borrower; owner can preview own resource.
My bookings / booking detail	Status, decision reason, queue position, cancellation/extension controls, start/end condition records, report damage/delay/suspected conflict action and Trust Record; user sees their own records.	Borrower sees own records; owner sees bookings for assigned resources.
Owner workbench	Own resources, pending queue, decision reason, supervisor assignment when required, approval/rejection/waitlist and handoff/return acknowledgement.	Owner and authorised managers.
Manager review	Pending affiliation/training/owner verification, incidents, queue/usage exceptions.	Manager; admin can inspect.
Admin settings / audit	Demo role assignment and visible fair-use settings; system event view.	Admin only; manager scope remains campus-only.
Notifications panel	Unread notices, due/overdue, reason and link to the relevant booking.	Each user sees own; operational managers see assigned alerts.

Use a shared booking-detail page that changes available actions by role. Do not build four separate dashboards, a chart-heavy landing page or bespoke UI for every status.

9. Build order and time budget

The window runs Wed 28 Oct 2026 10:00 IST to Fri 30 Oct 2026 10:00 IST. At the current review time, Wed 28 Oct 13:32 IST, about 44 h 28 min remain. This plan protects two sleep/recovery blocks and starts deployment early.

Time (IST)	Phase	Work	Exit gate
Wed 13:30–14:30	Bootstrap + first deploy	Create repo, Vite app, environment template, README stub; deploy a blank shell and verify the public URL.	Blank app builds, URL opens.
Wed 14:30–16:30	Schema, auth, roles, RLS	Create Supabase tables/migration, test accounts for all four roles, base RLS/grants, fictional seed data.	Signed-in resource list reads from DB; borrower cannot mutate role.
Wed 16:30–19:00	Resources and eligibility	Owner create/edit/publish/pause; equipment + space; search/filter/detail; manager verify affiliation/training.	Resource catalogue and eligibility blocker work.
Wed 19:00–22:00	Booking and owner decisions	Request submission, status, recheck on approval, reasoned reject, FIFO waitlist and database no-overlap.	First persisted request → approval/rejection path works.
Wed 22:00–22:45	Break + checkpoint	Commit, note blockers, confirm deployment still works.	Known-good checkpoint.
Wed 22:45–Thu 01:45	Handoff, return, Trust Record	Start condition, owner acknowledgement, end condition, owner confirms; create audit events and notices.	Core journey through return/history works before extras.
Thu 01:45–06:15	Sleep / recovery	Stop coding; keep clean committed state.	Avoid high-risk late-night regressions.
Thu 06:15–08:45	Fair access, cancel/extension	Usage caps, waitlist queue, reasoned overrides, cancellation and extension decision.	Conflict/fairness/cancel/extension scenarios pass.
Thu 08:45–09:15	Break	No feature changes.	Clear next-task list.
Thu 09:15–11:15	Incidents and notifications	Damage/delay reporting, human review/resolution, in-app notifications, due/overdue reminder function.	Incident closes with authorised review; notices persist.
Thu 11:15–12:00	Group usage + spaces regression	Verify group cap and a space lifecycle; check resource windows and conditions.	Brief coverage includes both equipment and spaces.
Thu 12:00–12:45	Meal / backup	Commit and back up.	Known-good build.
Thu 12:45–14:45	Security and business-rule tests	Direct table/RPC denied cases, overlapping reservations, role isolation, audit atomicity.	No critical authorization or double-booking defect.
Thu 14:45–16:45	Integration/regression	Run grouped acceptance scenarios; fix highest-severity defects.	Main journey repeats successfully after refresh.
Thu 16:45–18:00	Usability + docs outline	Mobile/laptop readability, labelled forms, status messages, README/test instructions.	A judge can navigate without narration.
Thu 18:00–23:00	Sleep / recovery	Stop coding.	Clean state for final tests.
Fri 00:00–02:30	End-to-end test + blocker fixes	Fresh browser, separate role accounts, conflict and ineligible cases.	All core release checks pass.
Fri 02:30–04:00	Deployment reminders and edge cases	Test due/overdue generation and deployed config; fix only important defects.	Production database and notices working.
Fri 04:00–05:00	Feature freeze	No new scope; only blockers/high severity.	Schema and feature list frozen.
Fri 05:00–06:30	Final report / disclosures	Official-format PDF, architecture, tests, features done/unfinished, limitations, tech/tools/reuse disclosures.	All required written artefacts complete.
Fri 06:30–07:30	Production smoke test	Redeploy if needed, verify URL, roles, DB, access, notifications and repository permissions.	Fresh-browser deployment passes.
Fri 07:30–08:30	Judge rehearsal	Repeat full workflow plus denied case from README only; check credentials and links.	No developer assistance required.
Fri 08:30–09:00	Submit early	Submit project link, repo, PDF, test steps, roles/contribution and disclosures.	Submission confirmation captured.

Time (IST)	Phase	Work	Exit gate
Fri 09:00–10:00	Contingency buffer	Only submission/link blocker fixes.	Deadline met with buffer.

10. Release checklist: 12 grouped scenarios

Test	Pass condition
T1 Listing and discovery	Verified owner can publish one equipment item and one space; fields/rules/location/availability and supervision flag are visible; search/filter works; unverified owners and paused resources cannot be requested.
T2 Profile, eligibility and supervision	Pending affiliation and missing/expired training show precise blockers; manager verification changes eligibility; owner cannot approve while it fails; a supervised resource cannot be approved until an authorised supervisor is assigned and confirmed.
T3 Request and decisions	Request persists with requester/resource/time/purpose/snapshot; owner can decide only on assigned resources; approval stores a clear explanation; rejection and waitlist require a human-readable reason; waitlist order is FIFO by default.
T4 Conflict integrity	Overlapping approved reservations cannot coexist, including concurrent attempts; adjacent intervals are allowed; if a checked-out item is overdue or return-pending, every new approval is blocked until actual release confirmation; conflicting user is told to choose another time or join waitlist.
T5 Fair access	Individual/group use caps and open-item limits match the visible policy; counts are consistent, FIFO is visible, and any exception requires actor + reason in history.
T6 Cancellation and extension	Cancellation releases slot and sends notice; extension request and decision are logged; approved extension rechecks overlap and caps.
T7 Start/handoff	Only an approved booking can start; equipment accessory/condition or space condition checklist and both-party handoff acknowledgement are required; active custody is retained until release confirmation.
T8 Return/condition mismatch	End condition persists; custodian confirms return/end-of-use before release; mismatch opens an incident; display label distinguishes equipment return from space end confirmation; overdue item remains unavailable until release confirmation.
T9 Incident review	Borrower/owner can report damage, delay or a suspected scheduling conflict; incident is linked to resource/booking, reviewed and resolved by an authorised owner/manager with a note; no automatic penalty; unrelated borrowers cannot view incident details.
T10 Notifications and audit	Request/decision/cancel/extension/incident create in-app notices; one 15-minute scheduled job creates a due-soon notice within 60 minutes of planned end and an overdue notice while unreturned; dedupe uses booking + type + planned end, so approved extensions create a new cycle. Resource publishing/pausing, verification and supervisor assignment are audited; failed actions have no success event.
T11 Authorization and privacy	Direct table/RPC tests fail for cross-user/private data, role escalation, owner-other-resource action and audit mutation; no service-role key in built assets.
T12 Submission-ready product	URL and test accounts work in a fresh private browser; persisted state survives refresh; keyboard navigation, visible focus, labelled fields and non-colour-only status cues are present; README enables full replay; report/disclosure/limits complete.

These 12 scenarios group the prior 20 tests; coverage is not dropped. Every official requirement in section 2 maps to at least one scenario above.

11. Risks and fallback decisions

Risk	Impact	Mitigation / cut rule
RLS or RPC allows wrong role	Critical	Write role matrix before policies; test direct API calls, not just hidden buttons. If broken, stop feature work and fix. Never make all tables public-write.
Double-booking race	Critical	Use database exclusion constraint; test conflict at approval time. Cut calendar visualisations, not the constraint.
Authentication and demo account setup consumes time	High	Create all fictional accounts early using Supabase Dashboard/admin workflow; do not put service keys in repo. No public role switcher.
Reminder scheduler unavailable	Medium	Configure the due-soon/overdue function early, test it directly and inspect Cron run history. If Supabase Cron fails, use a reliable scheduled invoker; a UI badge/manual call is not an acceptable replacement for the reminder requirement.
Scope creep	High	No AI, QR, photos, charts, payments, external messages, multi-campus or recurring slots before all release tests pass.
Deployed app gets modified by public demo use	Medium	Only synthetic data; protected manager/admin reset procedure if feasible. Do not expose unrestricted reset SQL/RPC.
Insufficient time / sleep	High	Protect sleep blocks; maintain commits/checkpoints. Cut style polish and optional analytics first.
Template/licence/disclosure issue	High	Use generic scaffolding and component packages; avoid cloning completed resource-sharing apps; keep source/version/licence/disclosure register.

If time gets tight, cut in this order

- Animations, theme switcher, custom charts, advanced calendar view and decorative landing sections.
- Advanced table sorting, complex filter combinations and utilisation dashboards.
- Optional space capacity/accessibility metadata that is not needed for the demo; preserve basic location/rules/accessibility notes where applicable.
- Any optional AI, QR/photo upload, email/SMS/push, external calendar or multi-campus feature (already out of MVP).

Never cut: complete lifecycle, explicit eligibility blockers, owner approval/rejection reasons, database no-overlap, fair-use policy and FIFO queue, initial/final condition records, incident with human review, in-app notices/reminders, role security, audit history, both resource types, test instructions and submission disclosures.

12. Code layout and implementation conventions

```
campusshare/
  README.md
  .env.example
  package.json
  src/
    app/              # routes, shared shell, role navigation
    components/       # form fields, cards, badges, dialogs
```

```
  features/
    auth/            # sign-in, demo credentials instructions
    resources/       # catalogue, detail, owner resource editor
    bookings/        # request, review, lifecycle actions
    eligibility/     # affiliation/training gate UI
    conditions/      # start/end checklists
    incidents/       # report, review, resolution
    notifications/   # inbox, due/overdue states
    trust-record/    # activity timeline
  lib/               # Supabase client, validators, date helpers
  types/             # role, status, API/domain types
supabase/
  migrations/        # schema, RLS, functions, triggers, constraints
  seed.sql           # fictional profile/resources/booking fixtures
  tests/             # DB policy/business-rule tests where practical
docs/                # test script, screenshots, report source
```

Create real Auth users through the Supabase Dashboard or a trusted admin workflow; SQL seed files must not require or contain a service-role secret. Link public profile rows to auth.users after the fictional accounts exist. Keep migrations as the reproducible source of schema/policies/functions.

Required repository documentation

- What problem and users the product serves; why the scope is a single-campus resource desk.
- Local setup, environment variable names, install/run/build commands and deployment URL.
- Architecture and a simple explanation of roles, eligibility, booking conflict and fairness rules.
- Judge accounts or credentials shared through the submission channel, exact demo path and seeded test cases.
- Testing results, known limitations, finished/unfinished feature list and future scope.
- Reuse disclosure: framework, templates/components, libraries, APIs, deployment/auth/database providers and significant AI/no-code tools.

13. Required submission package

The official statement requires a functioning product link, source repository, project PDF, test instructions/credentials, participant details and reuse disclosure. The PDF must explain the selected use case, users/owners, evidence or reasoning for why the problem matters, solution and complete workflow, features built/unfinished, stack and architecture, AI tools/APIs/frameworks/external services, privacy/security/fairness, limitations and future scope.

Submission item	Contents	Pass condition
Live product link	Public/judge-accessible URL, demo accounts/access instructions, no developer-only manual actions.	Fresh private-browser run completes the core workflow.
Source repository	Source, migrations, fictional seed data, README, setup/build/test/deploy instructions, no secrets.	Judges can inspect source and understand how to run it.
Official-format PDF	Use case; target users and owners; problem reasoning/evidence; proposed solution; complete workflow; features completed/unfinished; stack/architecture; AI/tools/APIs/external services; privacy/security/fairness; limitations; future scope.	Every required topic appears explicitly.
Test instructions and credentials	Borrower, owner, manager, admin demo accounts; exact main journey; eligibility/conflict/incident test; reset/replay notes if available.	Judges can follow steps without a walkthrough.
Participant/contribution	Solo developer details and actual role/contribution in first person.	Consistent with source history; no invented team members.

Submission item	Contents	Pass condition
Reuse and tool disclosure	Templates, UI components, libraries, framework, APIs, external services, deployment/auth/database platforms and significant AI/no-code tools; licence/source.	All actual reuse is listed and required licence notices retained.

14. Pre-build go/no-go checklist

☐ Official problem statement is treated as master; no conflict between product decisions and its explicit asks.

☐ Both equipment and spaces are represented using one coherent booking lifecycle.

☐ Four roles exist with server/database-enforced permission boundaries.

☐ Every required behaviour in section 2 has a visible, testable MVP implementation.

☐ No high-risk/regulated resources, real identity integration or sensitive institutional data.

☐ The plan uses one frontend stack, one hosted database/auth system and one deployment path.

☐ Generic scaffold/components are disclosed; no complete existing booking system is passed off as original work.

☐ Critical path is schema/security → resource → request/decision → fairness/conflict → handoff/return/incident/history → notifications → tests → submission.

☐ Feature freeze, report writing, judge rehearsal and a submission buffer are protected.

Final conclusion

The concept is approved. This final compatibility pass preserves every explicit requirement while tightening the supervision gate, conflict recovery, overdue-custody protection, reminder timing and consistent count-based fairness limits. Proceed with this v1.2 plan in the build stage; do not restart product ideation or add more features before the core workflow is running.

15. Research and technical references

- Official I-STEM overview - underused infrastructure, equipment discovery, availability, requests and host approval
- I-STEM official guidelines / end-to-end facility booking
- Supabase Auth with React quickstart
- Supabase Row Level Security - grants + policies; never expose service-role keys
- Supabase securing data guidance
- Supabase database functions / RPC
- Supabase Cron
- Supabase database testing
- PostgreSQL range types and exclusion constraints for non-overlapping reservations
- Render static-site deployment docs
- Resource Scheduler component reference
- MIT room booking reference (study patterns only)
- Shelf asset-management reference; AGPL-3.0 except separately licensed companion app
- LibreBooking resource scheduling reference

Research boundary: this is a focused product/technical review, not a full security audit of all dependencies, hosting terms or existing systems. The competitor review establishes adjacent capabilities and broad design lessons; it does not establish that competitors lack the specific workflow CampusShare will implement.